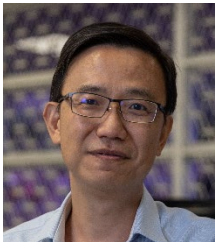


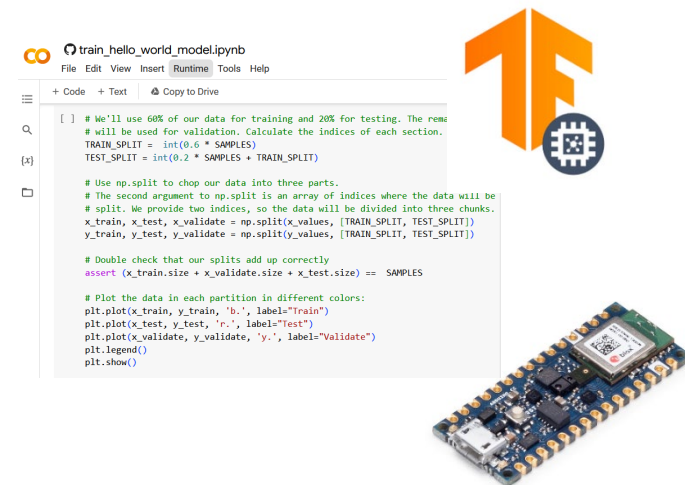


SC4172 Internet of Things - TinyML

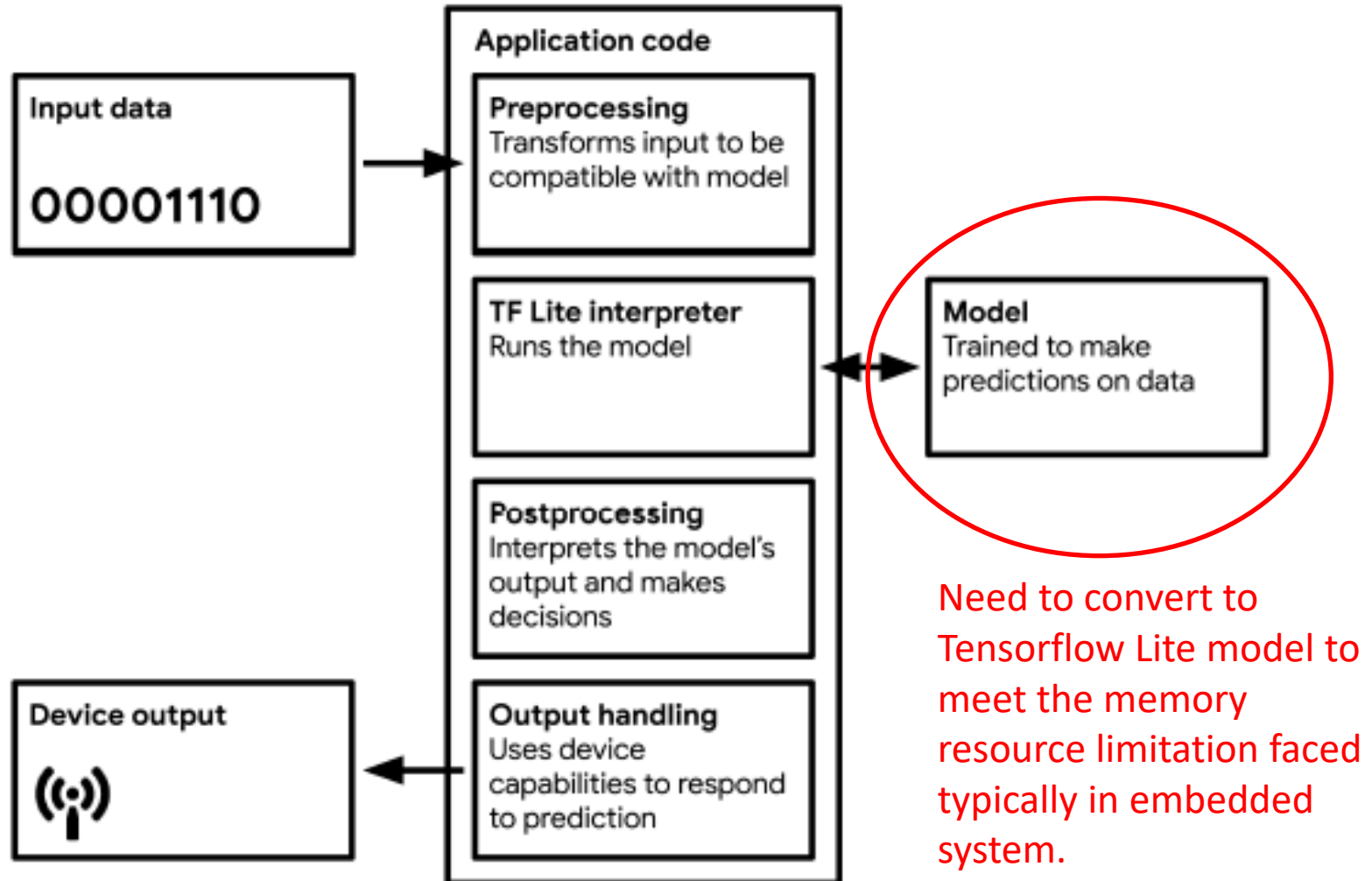
TensorFlow Lite, μ C Deployment, Gesture Recognition



Oh Hong Lye
Senior Lecturer
SCSE, Nanyang Technological University.
Email: hloh@ntu.edu.sg



A TinyML Application Architecture



TENSORFLOW LITE



TensorFlow Lite

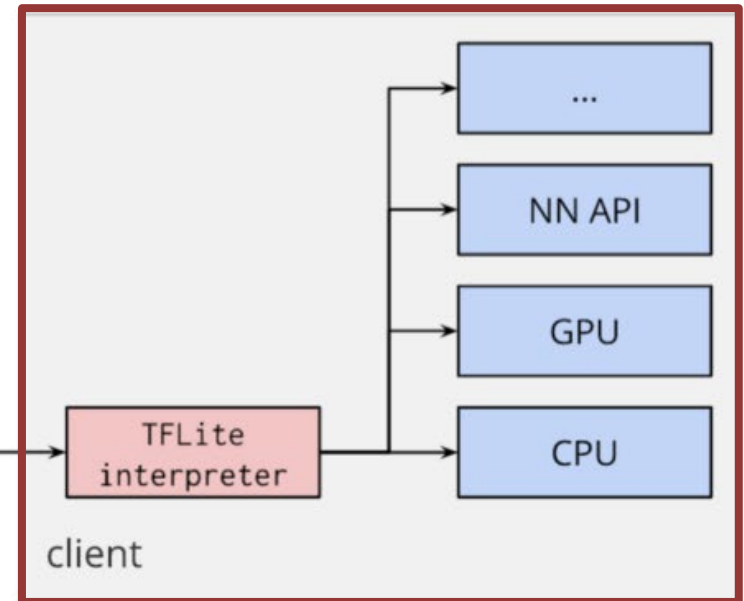
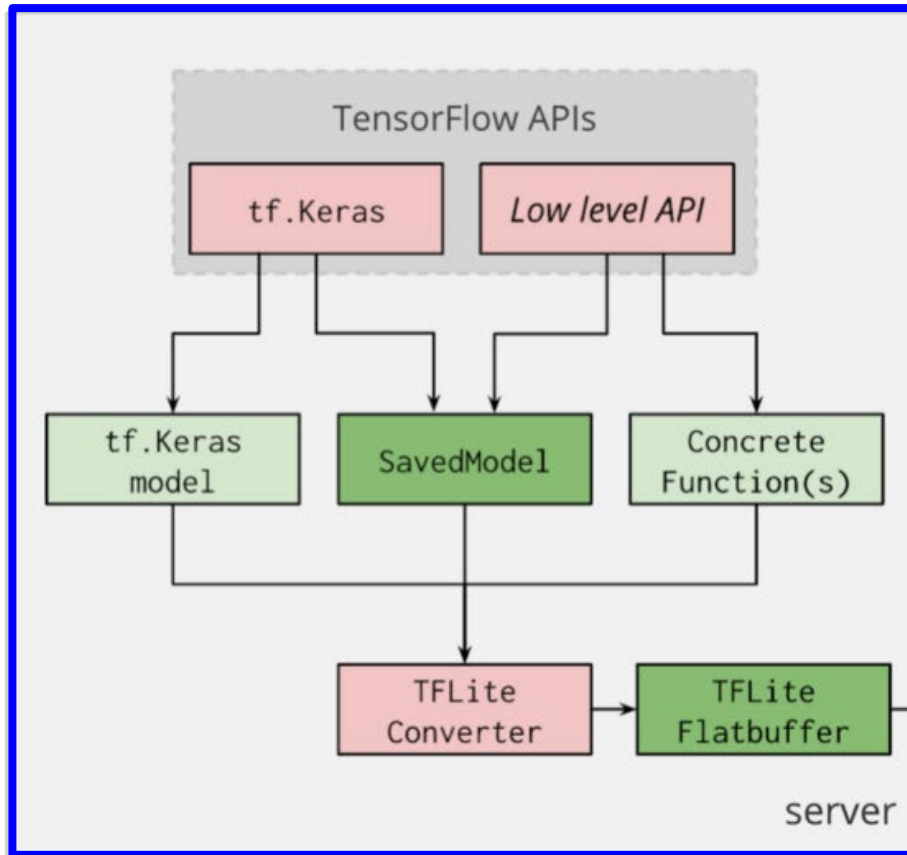
- TensorFlow Lite
 - a set of tools that enables on-device machine learning (Inference)
 - designed to deploy models efficiently on mobile, embedded, and IoT devices
 - optimizing for speed, model size and power consumption
- Size and complexity are reduced from the TensorFlow to fit resource constraint applications by dropping features that are less commonly needed
 - doesn't support training, only inference
 - doesn't support full range of data type (e.g. double)
 - doesn't include less-used operations
- TF Lite applications can fit within just a few hundred KB
 - Supports ~150 operations. TensorFlow core supports >1200 ops.
 - optimized library for ARM Cortex family of CPU

TensorFlow Lite

- TF Lite **converter** – to convert the TF model to a size-efficient format for memory constrained devices
 - **Flatbuffer** data format (serialised data)
 - load the **parts of the file** that need to read, reduces the overall memory footprint
 - access serialised data **without parsing/unpacking**
- Can further apply optimizations to reduce size and increase speed via **Quantization**,
- Many microcontrollers do not have **hardware** support for **floating point** operation
 - **Replacing 32-bit floating point with 8-bit integer** for Model's Weights results in
 - **smaller** size (4 bytes to 1 byte)
 - **faster** execution (fixed point arithmetic)
 - with **trade-off** in accuracy
- TF Lite models is executed by using the **TensorFlow Lite Interpreter**

TensorFlow Lite Architecture

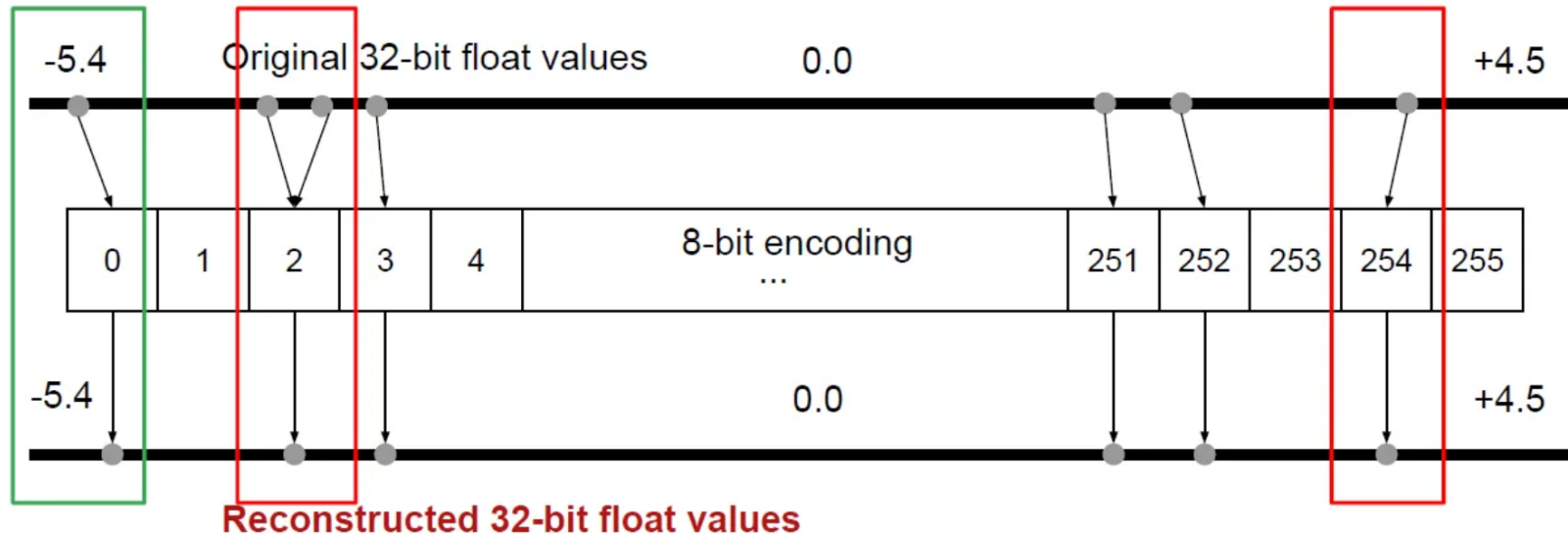
Desktop/Cloud



Embedded System

Source: www.tensorflow.org/lite

Quantization



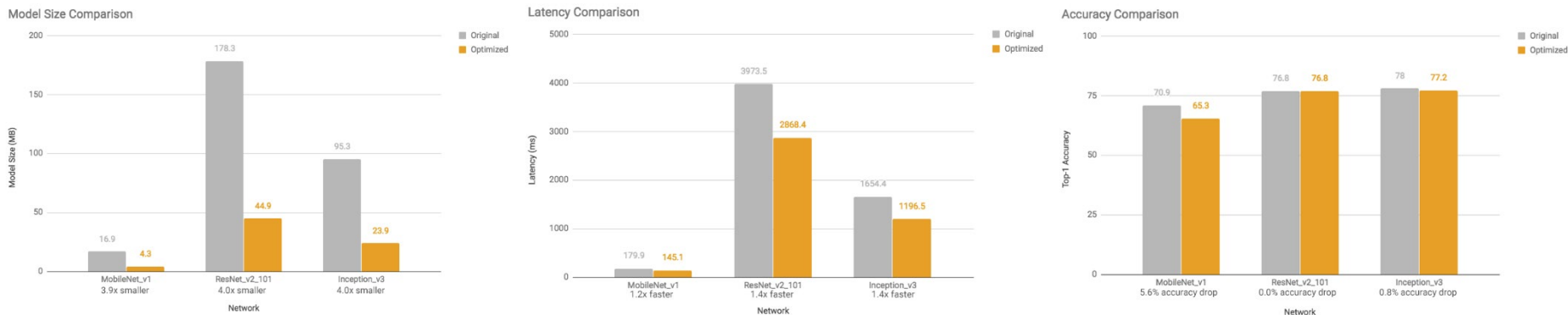
Source: medium.com

- Example: **Range** of **weights** in the model is between -5.4 to +4.5
- This can be **quantised** to **8-bit integer** 0-255.
- **Re-construction** to floating point can be done **if required**.
- **Many possible combinations** in leveraging Quantization as a memory/speed optimization option.
- **Trade-off** between memory/speed optimization and accuracy.
- More details in https://www.tensorflow.org/lite/performance/model_optimization

Post Training Quantization

- Up to 4x reduction in model sizes
- 10–50% faster execution for models which consist primarily of convolutional layers
- Up to 3x speed-up for RNN-based models
- With reduced memory and computation requirements, most models will also have lower power consumption

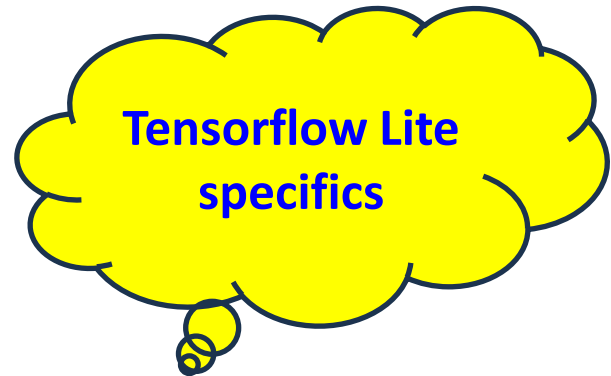
Original Optimised



[Source: https://blog.tensorflow.org/2018/09/introducing-model-optimization-toolkit.html](https://blog.tensorflow.org/2018/09/introducing-model-optimization-toolkit.html)

Workflow of Training and Building a Model

- The following are typical steps involved in developing a model based on machine learning (deep learning)
 - Decide on a goal
 - Select, collect and label a dataset
 - Design a model architecture
 - Train the model using the dataset
 - **Convert the model**
 - **Run the inference**
 - Evaluate and troubleshoot



Converting Model to TensorFlow Lite

- Converting the model (without quantization)

```
1 # Convert the model to the TensorFlow Lite format without quantization
2 converter = tf.lite.TFLiteConverter.from_keras_model(model_2)
3 tflite_model = converter.convert()
4
5 # Save the model to disk
6 open("sine_model.tflite", "wb").write(tflite_model)
```

- Quantization (Floating point to 8bit integer format)

```
1 # Convert the model to the TensorFlow Lite format with Dynamic quantization
2 converter = tf.lite.TFLiteConverter.from_keras_model(model_2)
3
4 # Set the optimization flag - DEFAULT: Float to Int quantization for Weights.
5 converter.optimizations = [tf.lite.Optimize.DEFAULT]
6
7 #convert the model
8 model_dyn_quant_tflite = converter.convert()
```



What's the default?

Converting Model to TensorFlow Lite with quantization

- Full Integer Quantization

- also quantize the activation values
- requires a calibration step to determine scaling parameters from a representative dataset.

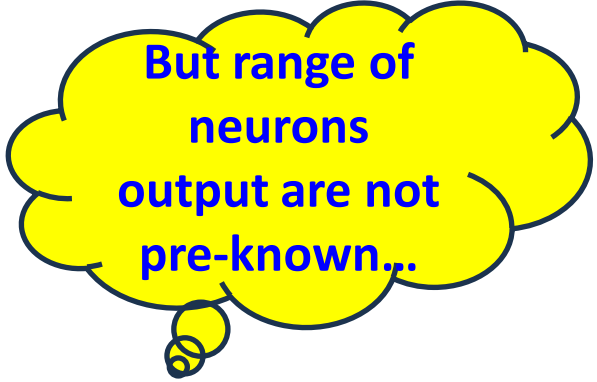
```
▶ # Define a generator function that provides our test data's x values
# as a representative dataset, and tell the converter to use it
def representative_dataset_generator():
    for value in x_test:
        yield [np.array(value, dtype=np.float32, ndmin=2)]
```

```
▶ # Convert the model to the TensorFlow Lite format with quantization
converter = tf.lite.TFLiteConverter.from_keras_model(model_2)
```

```
▶ # Indicate that we want to perform the default optimizations,
# which include quantization
converter.optimizations = [tf.lite.Optimize.DEFAULT]
```

```
▶ converter.representative_dataset = representative_dataset_generator
```

```
▶ # Convert the model to the TensorFlow Lite format without quantization
converter = tf.lite.TFLiteConverter.from_keras_model(model_2)
tflite_model = converter.convert()
# Save the model to disk
open("sine_model.tflite", "wb").write(tflite_model)
```



But range of
neurons
output are not
pre-known...

Running the TensorFlow Lite Interpreter

- Steps involved:
 - **Instantiate** an Interpreter
 - Call methods to **allocate memory** for the model
 - Write the input to the input tensor
 - Invoke the model
 - Read the output from the output tensor

```
❏ # Instantiate an interpreter for each model  
sine_model = tf.lite.Interpreter('sine_model.tflite')  
sine_model_quantized = tf.lite.Interpreter('sine_model_quantized.tflite')  
# Allocate
```

```
❏ # Allocate memory for each model  
sine_model.allocate_tensors()  
sine_model_quantized.allocate_tensors()
```

```
❏ # Get indexes of the input and output tensors  
sine_model_input_index = sine_model.get_input_details()[0]["index"]  
sine_model_output_index = sine_model.get_output_details()[0]["index"]  
sine_model_quantized_input_index = sine_model_quantized.get_input_details()[0]["index"]  
sine_model_quantized_output_index = sine_model_quantized.get_output_details()[0]["index"]
```

```
❏ # Create arrays to store the results  
sine_model_predictions = []  
sine_model_quantized_predictions = []
```

Running the TensorFlow Lite Interpreter

- Steps involved:
 - Instantiate an Interpreter
 - Call methods (i.e. function) to allocate memory for the model
 - Write the input to the **input tensor**
 - **Invoke** the model
 - Read the output from the **output tensor**

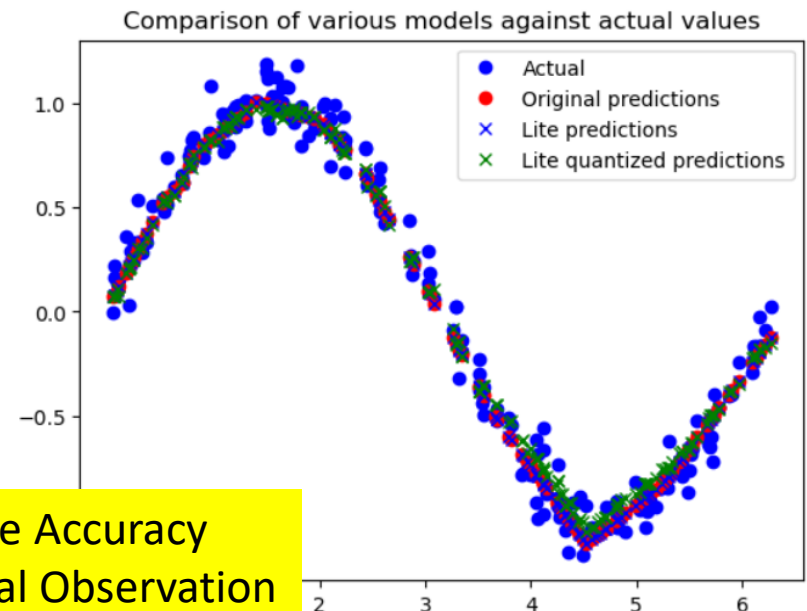
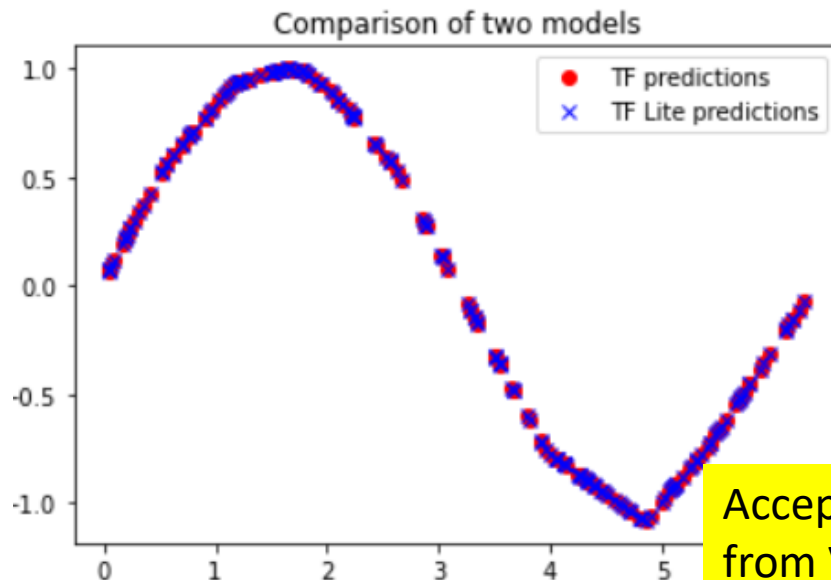
```
# Run each model's interpreter for each value and store the results in arrays
for x_value in x_test:
    x_value_tensor = tf.convert_to_tensor([[x_value]], dtype=np.float32)

    # Write the value to the input tensor
    sine_model.set_tensor(sine_model_input_index, x_value_tensor)
    # Run inference
    sine_model.invoke()
    # Read the prediction from the output tensor
    sine_model_predictions.append(sine_model.get_tensor(sine_model_output_index)[0])

    # Do the same for the quantized model
    sine_model_quantized.set_tensor(sine_model_quantized_input_index, x_value_tensor)
    sine_model_quantized.invoke()
    sine_model_quantized_predictions.append(sine_model_quantized.get_tensor(sine_model_quantized_output_index)[0])
```

Compare TF and TF Lite predicted outputs

```
▶ # See how they line up with the data
plt.clf()
plt.title('Comparison of various models against actual values')
plt.plot(x_test, y_test, 'bo', label='Actual')
plt.plot(x_test, predictions, 'ro', label='Original predictions')
plt.plot(x_test, sine_model_predictions, 'bx', label='Lite predictions')
plt.plot(x_test, sine_model_quantized_predictions, 'gx', \
label='Lite quantized predictions')
plt.legend()
plt.show()
```



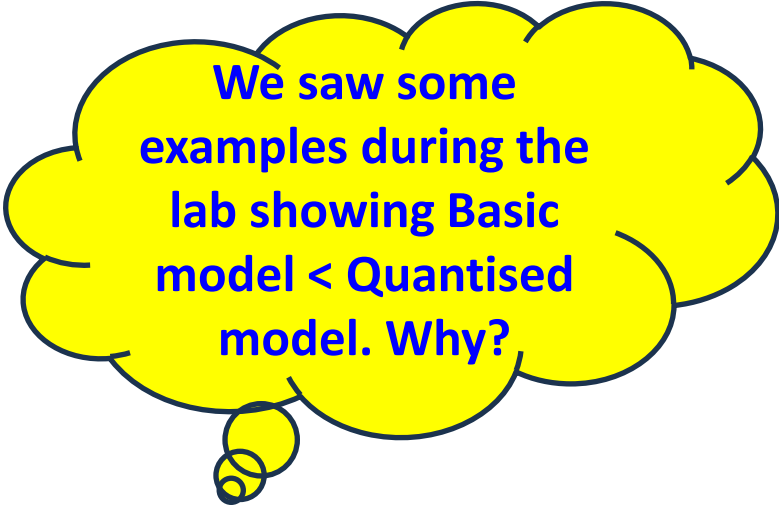
Acceptable Accuracy
from Visual Observation

TensorFlow Lite Model

Compare **model size** with and without **quantization**

```
▶ import os
basic_model_size = os.path.getsize("sine_model.tflite")
print("Basic model is %d bytes" % basic_model_size)
quantized_model_size = os.path.getsize("sine_model_quantized.tflite")
print("Quantized model is %d bytes" % quantized_model_size)
difference = basic_model_size - quantized_model_size
print("Difference is %d bytes" % difference)
```

Basic model is 3192 bytes
Quantized model is 3040 bytes
Difference is 152 bytes



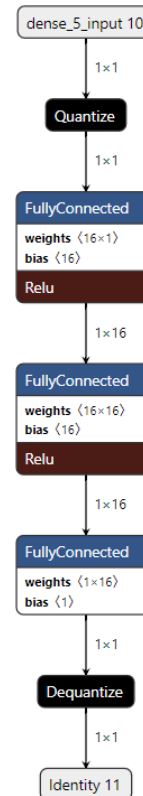
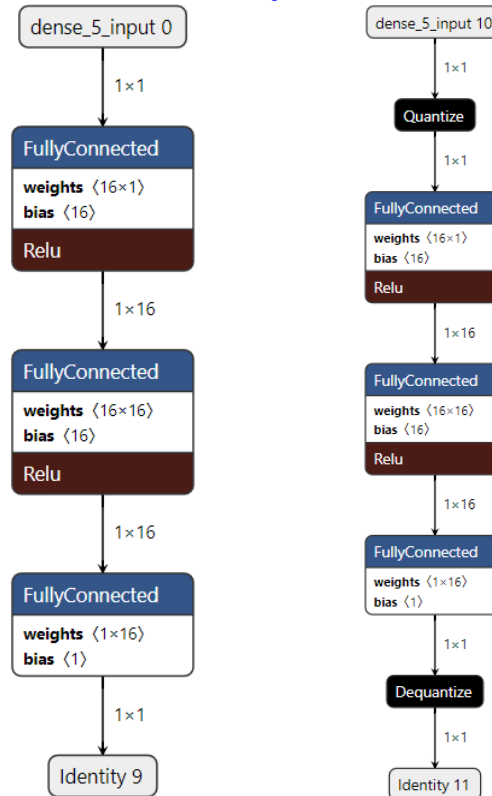
We saw some examples during the lab showing Basic model < Quantised model. Why?

Viewing TensorFlow Lite Model - Netron

Comparing models with and without **quantization** (of the **weights** and **biases**)

weights	name: sequential_2/dense_5/MatMul
	type: float32[16,1]
	location: 4
	[
	[
	0.09523486346006393
	],
	[
	0.23570004105567932
	],
	[
	0.272564172744751
	],
	[
	0.3017852008342743
	],
	[
	-0.27946001291275024
	],
	[
	
	]
	]

bias	name: sequential_2/dense_5/BiasAdd
	type: float32[16]
	location: 1
	[
	-0.15466751158237457,
	-0.6894649267196655,
	-0.6060609221458435,
	-0.4214448928833008,
	0,
	0,
	0,
	0,
	0,
	0,
	-0.8741070032119751,
	0.13749410212039948,
	-0.29287004470825195,
	-0.4120248258113861
	]



input	name: dense_5_input_int8
	type: int8[16,1]
	quantization: -0.3994057774543762 ≤ 0.0037136594764888287
	location: 4
	[
	[
	26
	],
	[
	63
	],
	[
	73
	],
	[
	81
	],
	[
	-75
	],
	]

bias	name: sequential_2/dense_5/BiasAdd/ReadVariableOp/resou
	type: int32[16]
	quantization: 0.00009134689753409475 * q
	location: 1
	[
	-1693,
	-7548,
	-6635,
	-4614,
	0,
	0,
	0,
	0,
	0,
	-9569,
	1505,
	-3206,
	-4511,
	0,
	8289,
	1759,
	]

Summary (TFLite)

- The **process** of building a model
 - collecting the **dataset**
 - deciding on the **model**
 - **train** the model
 - **evaluate** the performance – repeating from ii if needed
 - **convert** it to the **TF Lite model**, and **optimize** the model if needed
- We will next deploy the TF Lite model to run on a microcontroller

DEPLOYMENT TO MICROCONTROLLER



TensorFlow Lite For Microcontroller

- By this stage, we have
 - built and trained a model
 - checked its performance
 - converted the model into the TensorFlow Lite version
- All the above are done using Python programming language
- To deploy it to a microcontroller
 - need to convert the model into a C format source file
 - build an application that is integrated with the model
 - flash the application onto a Microcontroller
- In this example, we will be using the Arduino Sketch to develop and flash the program code onto the Arduino Nano 33 BLE Sense board. Arduino Sketch version used during testing was 2.3.3.
- The Tensorflow Lite Arduino library used is v2.4.0 ALPHA (available for download from NTU Learn).

Converting Model to C file for C/C++ program integration

- As the microcontroller **doesn't run OS** and has **no filesystem**, the model needs to be in a form that can be included in any C/C++ program
- For **TinyML** implementation, the model is stored in an **array of bytes** (in binary) **compatible** with C++ based applications
- On **Linux** machine, this can be done by using the tool named **xxd** as follows:

```
# Install xxd if it is not available
```

```
!apt-get -qq install xxd
```

```
# Save the file as a C source file
```

```
!xxd -i sine_model_quantized.tflite > sine_model_quantized.cc
```

```
# Print the source file
```

```
!cat sine_model_quantized.cc
```

Converting Model to C file – Python Code Method

- We can also use a [Python code](#) to generate a C code

```
1 # Function: Convert some hex value into an array for C programming
2 def hex_to_c_array(hex_data, var_name):
3
4     c_str = ''
5
6     # Create header guard
7     c_str += '#ifndef ' + var_name.upper() + ' H\n'
12
13     # Declare C variable
14     c_str += 'unsigned char ' + var_name + '[] = {'
15     hex_array = []
16     for i, val in enumerate(hex_data) :
17
18         # Construct string from hex
19         hex_str = format(val, '#04x')
20
21         # Add formatting so each line stays within 80 characters
22         if (i + 1) < len(hex_data):
23             hex_str += ','
24         if (i + 1) % 12 == 0:
25             hex_str += '\n '
26         hex_array.append(hex_str)
27
28     # Add closing brace
29     c_str += '\n ' + format(' '.join(hex_array)) + '\n};\n\n'
30
31     # Close out header guard
32     c_str += '#endif //' + var_name.upper() + ' _H'
33
34     return c_str
```

```
hello_world  arduino_constants.cpp  arduino_main.cpp  arduino_outp
#include "model.h"

// Keep model aligned to 8 bytes to guarantee aligned 64-bit
alignas(8) const unsigned char g_model[] = {
    0x1c, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, 0x00, 0x
    0x1c, 0x00, 0x04, 0x00, 0x08, 0x00, 0x0c, 0x00, 0x10, 0x
    0x00, 0x00, 0x18, 0x00, 0x12, 0x00, 0x00, 0x00, 0x03, 0x
    0x60, 0x09, 0x00, 0x00, 0xa8, 0x02, 0x00, 0x00, 0x90, 0x
    0x3c, 0x00, 0x00, 0x00, 0x04, 0x00, 0x00, 0x00, 0x01, 0x
    0x0c, 0x00, 0x00, 0x00, 0x08, 0x00, 0x0c, 0x00, 0x04, 0x
    0x08, 0x00, 0x00, 0x00, 0x08, 0x00, 0x00, 0x00, 0x0b, 0x
    0x13, 0x00, 0x00, 0x00, 0x6d, 0x69, 0x6e, 0x5f, 0x72, 0x
    0x69, 0x6d, 0x65, 0x5f, 0x76, 0x65, 0x72, 0x73, 0x69, 0x
    0x0c, 0x00, 0x00, 0x00, 0x48, 0x02, 0x00, 0x00, 0x34, 0x
    0x0c, 0x02, 0x00, 0x00, 0xfc, 0x00, 0x00, 0x00, 0xac, 0x
    0x8c, 0x00, 0x00, 0x00, 0x3c, 0x00, 0x00, 0x00, 0x34, 0x
    0x2c, 0x00, 0x00, 0x00, 0x24, 0x00, 0x00, 0x00, 0x1c, 0x
    0x04, 0x00, 0x00, 0x00, 0xfe, 0xfd, 0xff, 0xff, 0x04, 0x
    0x05, 0x00, 0x00, 0x00, 0x31, 0x2e, 0x35, 0x2e, 0x30, 0x
    0x7c, 0xfd, 0xff, 0xff, 0x80, 0xfd, 0xff, 0xff, 0x84, 0x
    0x88, 0xfd, 0xff, 0xff, 0x22, 0xfe, 0xff, 0xff, 0x04, 0x
    0x40, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfc, 0x
```

Converting Model to C file – Python Code Method

- We then call the Python Script as follows

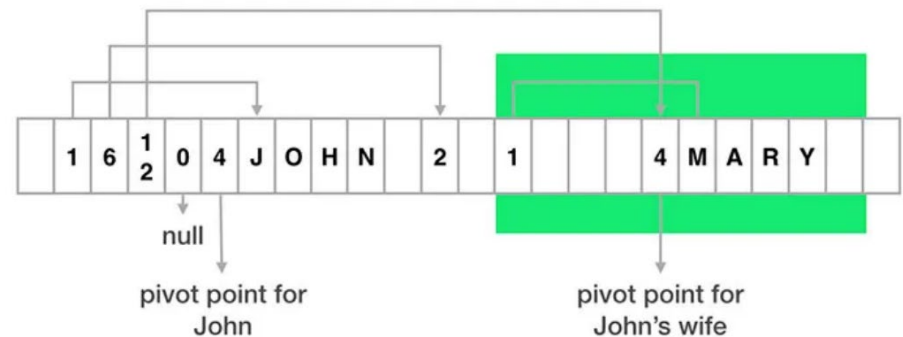
```
▶ # Write TFLite model to a C source file
  with open("sine_model_1.h", 'w') as file:
      file.write(hex_to_c_array(tflite_model, "sine_model_1"))
```

- This will create the C file, in the form of a header file with extension .h
 - sine_model_1.h
- We will then include this as a **header file** in the application programme to deploy on the Microcontroller

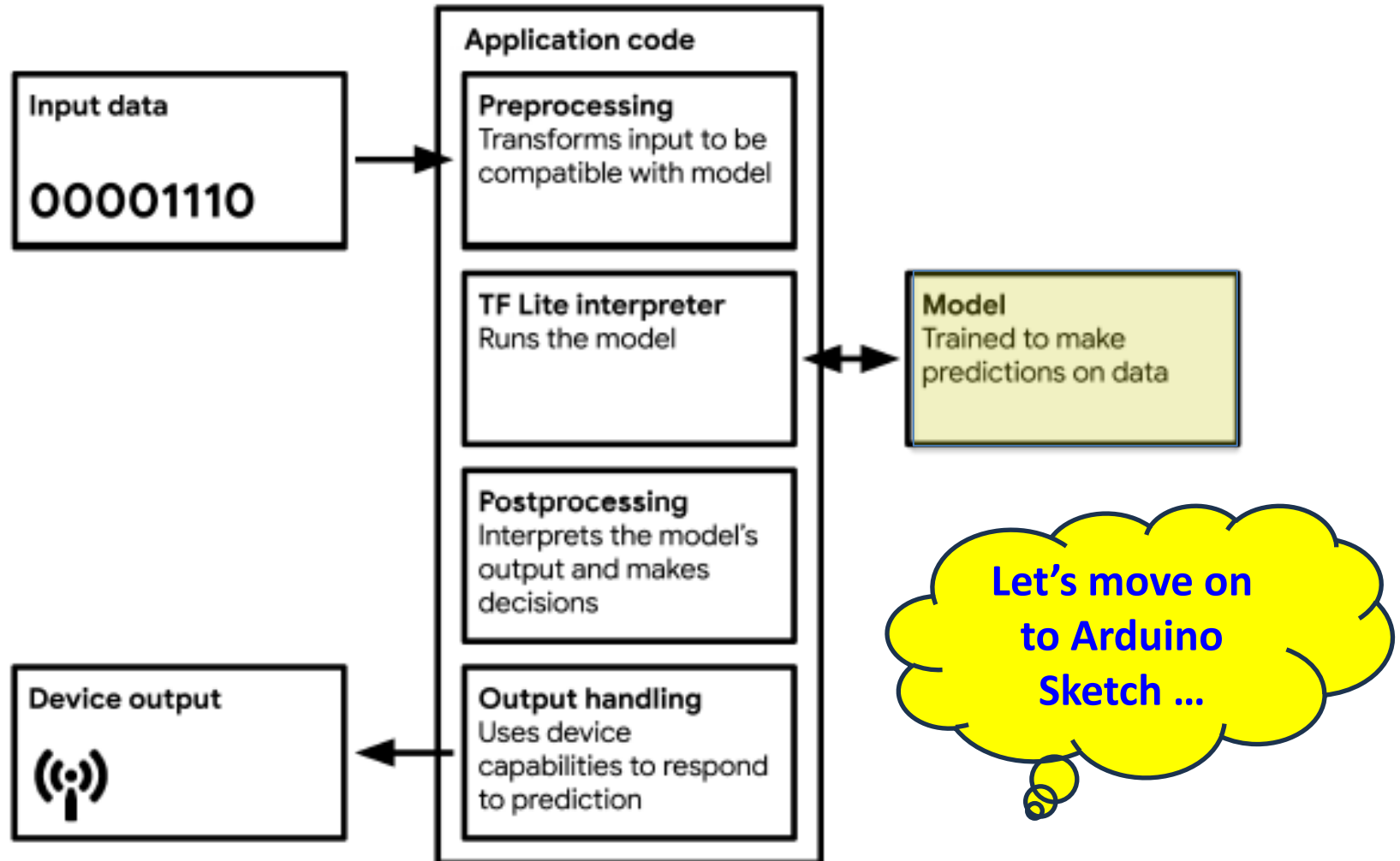
Format of the C header file Generated

```
1  #ifndef SINE_MODEL_1_H
2  #define SINE_MODEL_1_H
3
4
5  unsigned int sine_model_1_len = 3040;
6  unsigned char sine_model_1[] = {
7    0x1c, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, 0x14, 0x00, 0x20, 0x00,
8    0x1c, 0x00, 0x18, 0x00, 0x14, 0x00, 0x10, 0x00, 0x0c, 0x00, 0x00, 0x00,
9    0x08, 0x00, 0x04, 0x00, 0x14, 0x00, 0x00, 0x00, 0x1c, 0x00, 0x00, 0x00,
10   0x88, 0x00, 0x00, 0x00, 0xe0, 0x00, 0x00, 0x00, 0xac, 0x03, 0x00, 0x00,
11   0xbc, 0x03, 0x00, 0x00, 0x58, 0x0b, 0x00, 0x00, 0x03, 0x00, 0x00, 0x00,
256   0x0c, 0x00, 0x10, 0x00, 0x0f, 0x00, 0x00, 0x00, 0x08, 0x00, 0x04, 0x00,
257   0x0c, 0x00, 0x00, 0x00, 0x09, 0x00, 0x00, 0x00, 0x04, 0x00, 0x00, 0x00,
258   0x00, 0x00, 0x00, 0x09, 0x0c, 0x00, 0x0c, 0x00, 0x0b, 0x00, 0x00, 0x00,
259   0x00, 0x00, 0x04, 0x00, 0x0c, 0x00, 0x00, 0x00, 0x72, 0x00, 0x00, 0x00,
260   0x00, 0x00, 0x00, 0x72
261 };
262
263 #endif //SINE_MODEL_1_H
```

Flatbuffer Format Sample



A TinyML Application Architecture



(Source: TinyML Pete Warden & Daniel Situnayake)

Embedded Device Code for TinyML

- A typical TinyML program is coded in C/C++, it starts with the `#include` directive to import the various relevant header files
 - `#include "TensorFlowLite.h"`
 - `#include "tensorflow/lite/micro/kernels/micro_ops.h"`
 - `#include "tensorflow/lite/micro/micro_error_reporter.h"`
 - `#include "tensorflow/lite/micro/micro_interpreter.h"`
 - `#include "tensorflow/lite/micro/all_ops_resolver.h"`
 - `#include "tensorflow/lite/version.h"`
 - `#include "sine_model_1.h"`

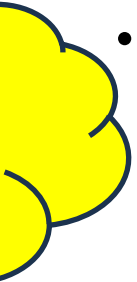
May be replaced by

`#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"`

More details later ...

TensorFlow Lite Header Files for Microcontroller

- `#include "tensorflow/lite/micro/micro_interpreter.h"`
 - This is the Tensorflow Lite for microcontroller interpreter which run the model
 - only support 50 operations
- `#include "tensorflow/lite/micro/all_ops_resolver.h"`
 - This contains ALL the operations supported by tensorflow lite
 - If `micro_mutable_op_resolver.h` is used instead, then only the registered ops are included (smaller memory size)
- `#include "tensorflow/lite/version.h"`
 - For checking the correct version of the model
- `#include "sine_model_1.h"`
 - This is the sine model that we trained, as shown in previous slide



AllOpsResolver vs MicroMutableOpResolver

- AllOpsResolver. Include ALL operations for TFLite.
- MicroMutableOpResolver. Include ONLY operations needed (need to register in the code).
- Full list of support operations for Tensorflow Lite
https://www.tensorflow.org/lite/guide/op_select_allowlist

229440 bytes (23%) of program storage space

AllOpsResolver

```
Library Arduino_TensorFlowLite has been declared precompiled:  
Using precompiled library in C:\Users\hloh\Documents\Arduino\libraries\Arduino_TensorFlowLite\src\cortex-m4\fpv4-sp-d16-softfp  
Sketch uses 229440 bytes (23%) of program storage space. Maximum is 983040 bytes.  
Global variables use 54368 bytes (20%) of dynamic memory, leaving 207776 bytes for local variables. Maximum is 262144 bytes.
```

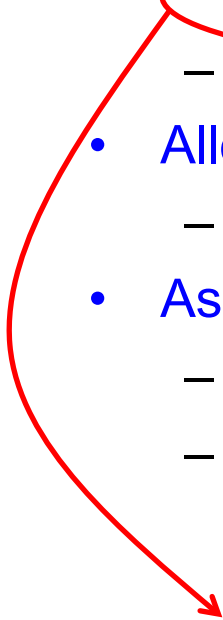
118144 bytes (12%) of program storage space

MicroMutableOpResolver

```
Library Arduino_TensorFlowLite has been declared precompiled:  
Using precompiled library in C:\Users\hloh\Documents\Arduino\libraries\Arduino_TensorFlowLite\src\cortex-m4\fpv4-sp-d16-softfp  
Sketch uses 118144 bytes (12%) of program storage space. Maximum is 983040 bytes.  
Global variables use 53016 bytes (20%) of dynamic memory, leaving 209128 bytes for local variables. Maximum is 262144 bytes.
```

Application Setup

- Create an Interpreter
 - ~~tflite::MicroInterpreter static_interpreter(model, resolver, tensor_arena, kTensorArenaSize, error_reporter);~~
 - ~~interpreter = &static_interpreter;~~
- Allocate memory from the Tensor Arena for the model's tensors
 - interpreter->AllocateTensors();
- Assign the input and output tensors buffers for the interpreter
 - model_input = interpreter->input(0);
 - model_output = interpreter->output(0);



Static memory allocation recommended for embedded system (limited memory resources).
May need some trial and error to get the optimal size.

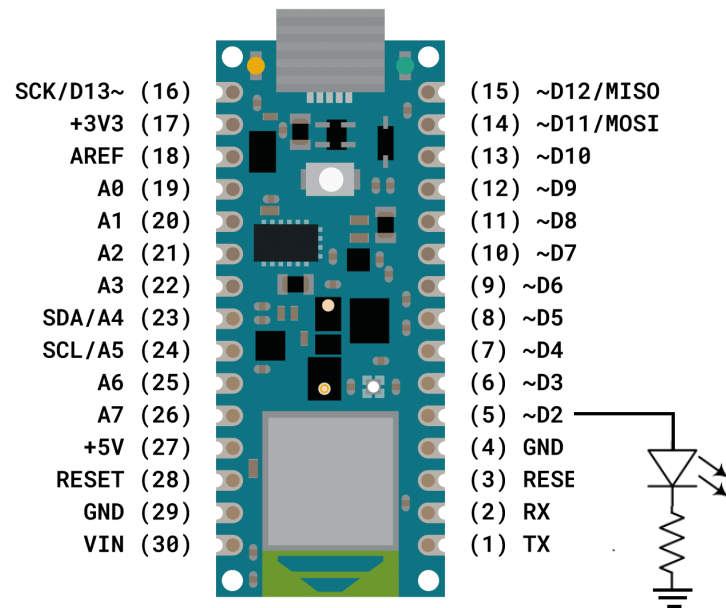
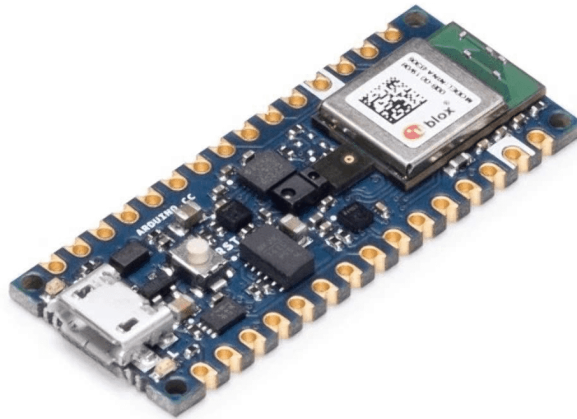
Application Execution

- After the setup (i.e. **initialization**) we can run the **inference**
- **Copy** the **data** into the **input tensor buffer**
`model_input->data.f[0] = x_val;`
- Run the model on the input to **perform inference**
`TfLiteStatus invoke_status = interpreter->Invoke();`
- and **check** the it is invoked successfully

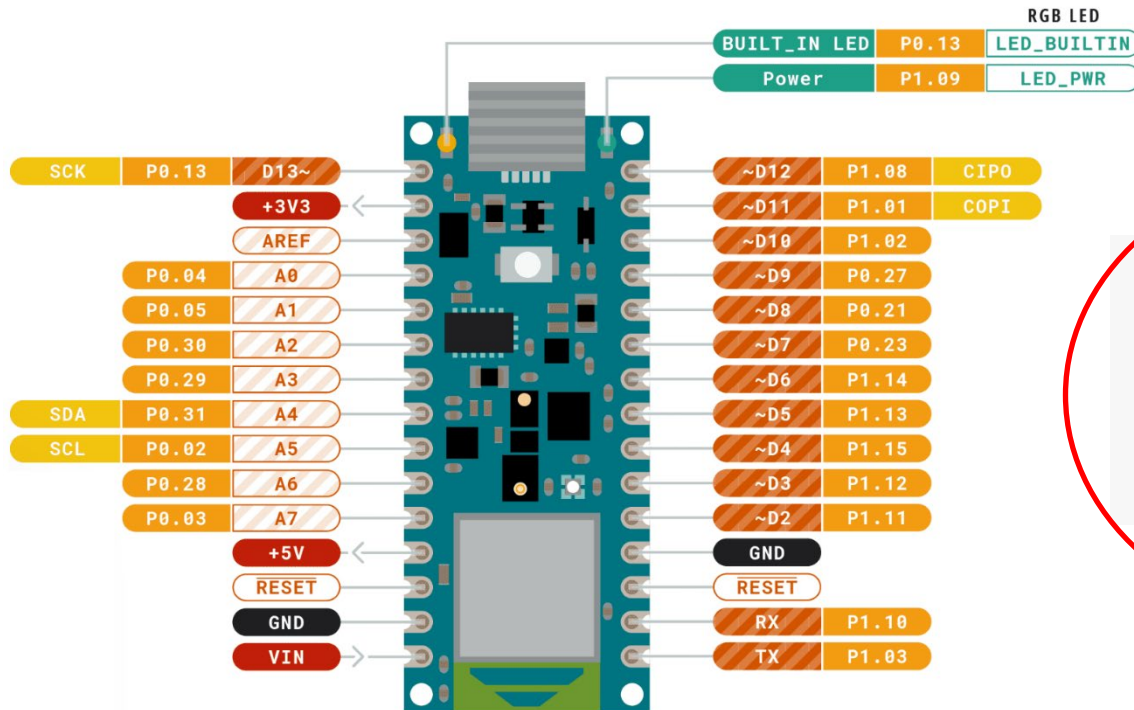
```
if (invoke_status != kTfLiteOk) {  
    error_reporter->Report("Invoke failed on input: %f\n", x_val);  
}
```
- **Read the output**
`y_val = model_output->data.f[0];`

Nano 33 BLE Sense Board

- The 'Hello World' program will use the [sine model](#) output to control the brightness of an [LED](#), which is connect to [pin D2](#) of the Arduino Nano 33 BLE Sense microcontroller board



Deploying to Microcontroller



```
#define RED 22
#define BLUE 24
#define GREEN 23
#define LED_PWR 25
```

- Pin number has some **abstraction** applied so **doesn't tally** with the **actual pin number** on the chip.
- Refer to [pins_arduino.h](https://www.arduino.cc/en/Reference/PinsArduino) for details on pin name to pin number matching in Arduino

Sample capture from pins_arduino.h

```
#define NUM_DIGITAL_PINS      (21u)
#define NUM_ANALOG_INPUTS    (8u)
#define NUM_ANALOG_OUTPUTS   (0u)

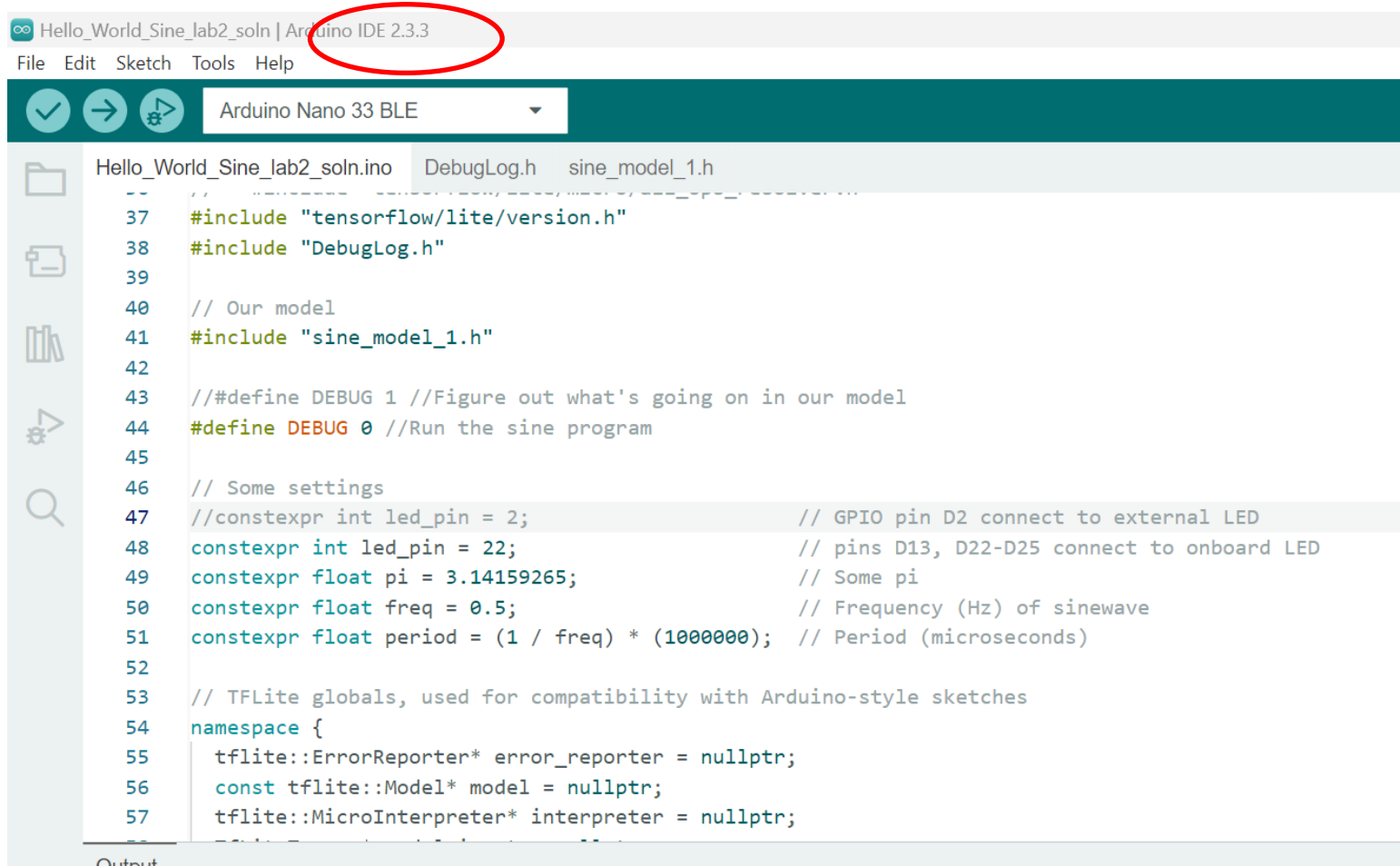
extern PinName digitalPinToPinName(pin_size_t P);

// LEDs
// ----
#define PIN_LED      (13u)
#define LED_BUILTIN PIN_LED
#define LEDR         (22u)
#define LEDG         (23u)
#define LEDB         (24u)
#define LED_PWR      (25u)

// Analog pins
// -----
#define PIN_A0 (14u)
#define PIN_A1 (15u)
#define PIN_A2 (16u)
#define PIN_A3 (17u)
#define PIN_A4 (18u)
#define PIN_A5 (19u)
#define PIN_A6 (20u)
#define PIN_A7 (21u)
static const uint8_t A0 = PIN_A0;
static const uint8_t A1 = PIN_A1;
static const uint8_t A2 = PIN_A2;
static const uint8_t A3 = PIN_A3;
static const uint8_t A4 = PIN_A4;
static const uint8_t A5 = PIN_A5;
static const uint8_t A6 = PIN_A6;
static const uint8_t A7 = PIN_A7;
#define ADC_RESOLUTION 12
```


Arduino Sketch

- The program is coded using the [Arduino Sketch IDE \(2.3.3\)](#)



```
37 #include "tensorflow/lite/version.h"
38 #include "DebugLog.h"
39
40 // Our model
41 #include "sine_model_1.h"
42
43 // #define DEBUG 1 // Figure out what's going on in our model
44 #define DEBUG 0 // Run the sine program
45
46 // Some settings
47 //constexpr int led_pin = 2; // GPIO pin D2 connect to external LED
48 constexpr int led_pin = 22; // pins D13, D22-D25 connect to onboard LED
49 constexpr float pi = 3.14159265; // Some pi
50 constexpr float freq = 0.5; // Frequency (Hz) of sinewave
51 constexpr float period = (1 / freq) * (1000000); // Period (microseconds)
52
53 // TFLite globals, used for compatibility with Arduino-style sketches
54 namespace {
55     tflite::ErrorReporter* error_reporter = nullptr;
56     const tflite::Model* model = nullptr;
57     tflite::MicroInterpreter* interpreter = nullptr;
```

Code snippets of the program

```
// Define settings to be used
```

```
constexpr int led_pin = 22;           // RED LED connects to pin 22
```

```
constexpr float pi = 3.14159265;     // pi value
```

```
constexpr float freq = 0.5;          // Frequency (Hz) of sinewave
```

```
constexpr float period = (1 / freq) * (1000000); // Period (uSec)
```

```
// LEDs
```

```
// ----
```

```
#define PIN_LED      (13u)
```

```
#define LED_BUILTIN PIN_LED
```

```
#define LEDR         (22u)
```

```
#define LEDG         (23u)
```

```
#define LEDB         (24u)
```

```
#define LED_PWR      (25u)
```

```
timestamp = timestamp % (unsigned long)period;
```

```
// Calculate x value to feed to the model and convert to radian
```

```
float x_val = ((float)timestamp * 2 * pi) / period;
```

LED brightness control

- The **brightness** of the **LED** is controlled by using **Pulse Width Modulation**

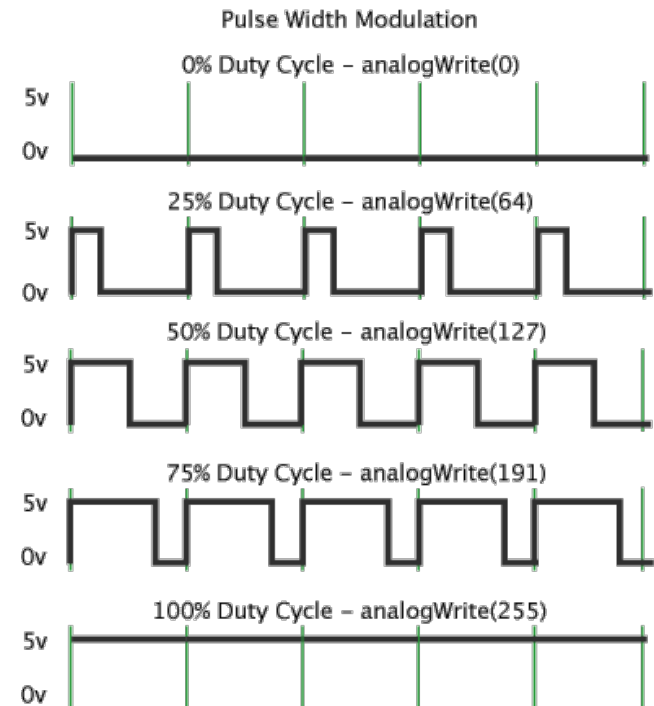
```
// Read predicted y value from output buffer (tensor)  
float y_val = model_output->data.f[0];
```

```
// Translate to a PWM LED brightness  
int brightness = (int)(255 * y_val);
```

```
// Output to the pin to drive the LED  
analogWrite(led_pin, brightness);
```

ADC or
DAC?

Resolution?



Serial Monitor Communications

- Debugging information are sent to the serial port using the Serial.print() function

```
Serial.println(y_val);
```

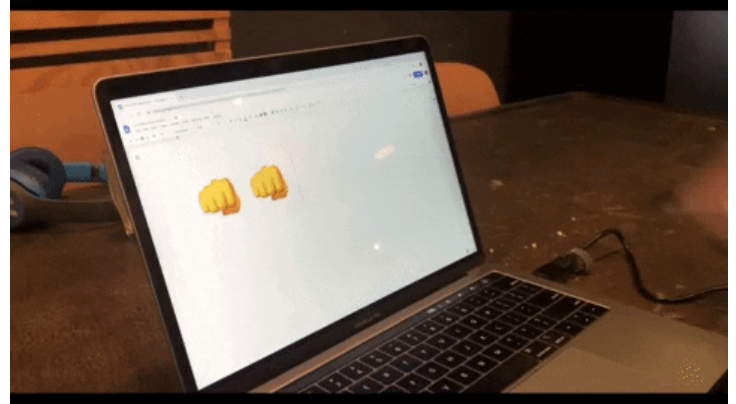
```
Serial.print("Time to execute inference (us): ");
```

```
Serial.println(micros() - start_timestamp);
```



Current Timestamp

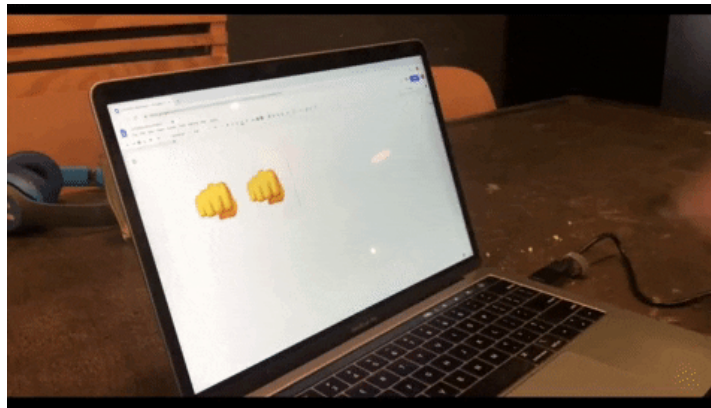
Timestamp at start of each
iteration of the loop()



GESTURE RECOGNITION (RELATED TO LAB3)

Gesture Recognition

- The Arduino Nano board 33 BLE Sense board also contain a **9-axis IMU** (**accelerator**, **gyroscope** and **magnetometer**) sensor which we can use to detect the movement that it is attached to
- E.g., we can held it in our hand and use it to detect the hand moment and hence recognize the different gestures



<https://blog.tensorflow.org/2019/11/how-to-get-started-with-machine.html>

Collection of Data for Training

- Training
 - Collect the data from the device
- This is to be done by capturing the sensor's output data
 - Send through the [USB](#) port that power the board
- An Arduino program is hence needed
 - [IMU_Capture.ino](#)
- Note: the library to access the sensor is to be installed through the Arduino Manager: [Arduino_LSM9DS1](#)

IMU_Capture program logic

- The logic of the program is to perform the following:
 - Monitor the board's accelerometer and gyroscope
 - Trigger a sample window on detecting significant linear acceleration of the board
 - Sample for one second at 119Hz, outputting CSV format data over USB
 - Loop back and monitor for the next gesture

IMU_Capture Code snippet

```
#include <Arduino_LSM9DS1.h>

const float accelerationThreshold = 2.5;
// threshold of significant in G's
const int numSamples = 119;

int samplesRead = numSamples;

void setup() {
  Serial.begin(9600);

  IMU.begin();

  // print the header to the USB port
  Serial.println("aX,aY,aZ,gX,gY,gZ");
}
```

IMU_Capture Code snippet (cont.)

```
void loop() {  
    float aX, aY, aZ, gX, gY, gZ;  
  
    // wait for significant motion  
    while (samplesRead == numSamples) {  
        if (IMU.accelerationAvailable()) {  
            // read the acceleration data  
            IMU.readAcceleration(aX, aY, aZ);  
  
            // sum up the absolutes  
            float aSum = fabs(aX) + fabs(aY) + fabs(aZ);  
  
            // check if it's above the threshold  
            if (aSum >= accelerationThreshold) {  
                // reset the sample read count  
                samplesRead = 0;  
                break;  
            }  
        }  
    }  
}
```

IMU_Capture Code snippet (cont.)

```
// check if the all the required samples have been read since
// the last time the significant motion was detected
while (samplesRead < numSamples) {
    // check if both new acceleration and gyroscope data is
    // available
    if (IMU.accelerationAvailable() && IMU.gyroscopeAvailable()) {
        // read the acceleration and gyroscope data
        IMU.readAcceleration(aX, aY, aZ);
        IMU.readGyroscope(gX, gY, gZ);

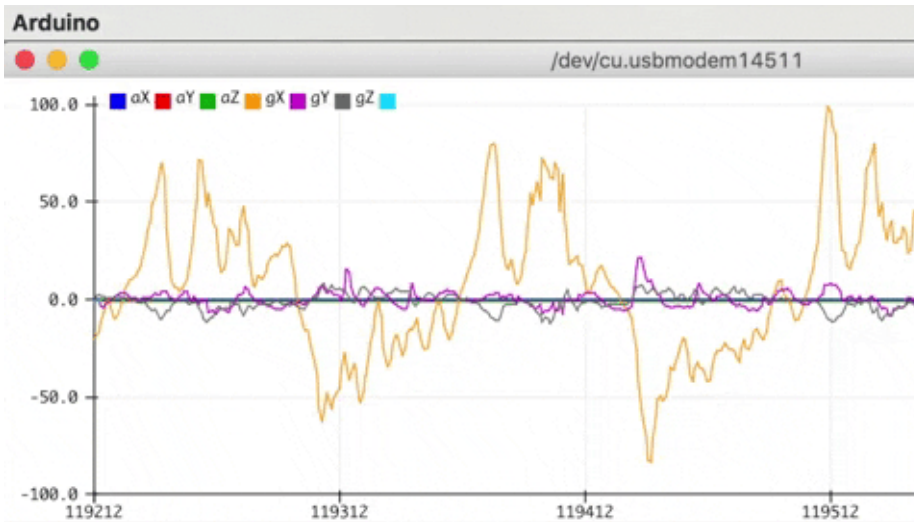
        samplesRead++;

        // print the data in CSV format
        Serial.print(aX, 3);
        Serial.print(',');
        Serial.print(aY, 3);
        ... ..
        Serial.print(gZ, 3);
        Serial.println();

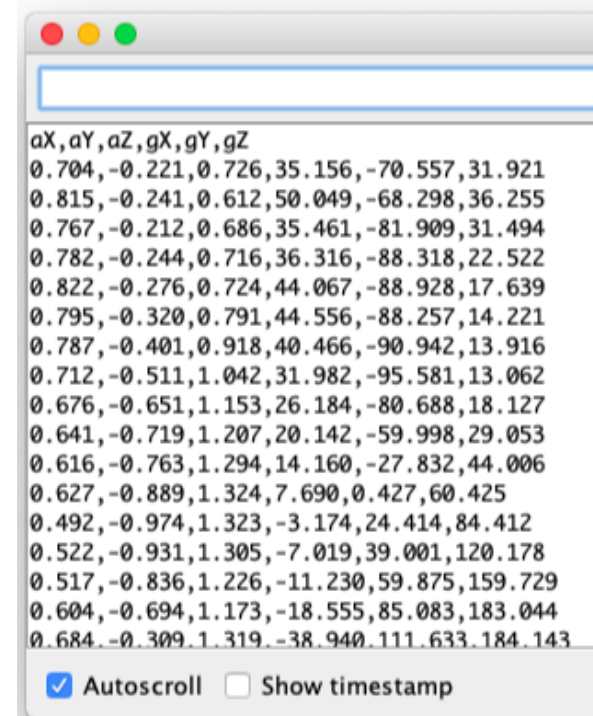
        if (samplesRead == numSamples) {
            // add an empty line if it's the last sample
            Serial.println();
        }
    }
}
```

Data Display

Serial plotter



Serial Monitor



Collecting the data

- Data can be copied directly from the Serial Monitor output and save into a file in the CSV format
- Procedure:
 - Make a punch gesture with the board in your hand
 - make the outward punch quickly enough to trigger the capture
 - Return to a neutral position slowly so as not to trigger the capture again
 - Repeat the gesture capture step 10 or more times to gather more data
 - Copy and paste the data from the Serial Monitor to a new text file called punch.csv

Collecting the data (cont.)

- Clear the console window output.
- Procedure:
 - Make a **flex** gesture with the board in your hand
 - make the **inward flex quickly** enough to **trigger** the capture
 - Return to a **neutral** position **slowly** so as not to trigger the capture again
 - **Repeat** the gesture capture step 10 or more times to gather more data
 - **Copy and paste** the data from the Serial Monitor to a new text file called flex.csv
- The two files, **punch.csv** and **flex.csv** will be used to **train** the **gesture recognition model**

Sample csv file

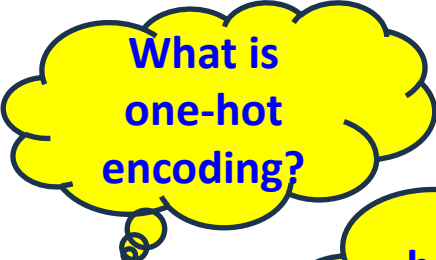
	A	B	C	D	E	F	G
1	aX	aY	aZ	gX	gY	gZ	
2	-1.048	-0.13	0.528	-36.743	37.17	-5.432	
3	-1.054	-0.122	0.566	-41.504	35.461	-7.019	
4	-1.028	-0.154	0.631	-45.715	33.203	-9.094	
5	-1.026	-0.199	0.67	-49.866	30.029	-11.353	
6	-1.021	-0.234	0.702	-58.96	28.687	-11.23	
7	-1.002	-0.254	0.721	-71.167	30.457	-8.057	
8	-1.021	-0.286	0.711	-83.313	31.555	-3.906	
9	-1.034	-0.314	0.704	-95.398	33.02	-0.732	
10	-1.054	-0.328	0.643	-107.727	33.447	0.977	
11	-1.051	-0.347	0.601	-118.652	34.424	1.831	
12	-1.044	-0.371	0.512	-127.686	28.992	1.709	
13	-1.056	-0.372	0.405	-135.01	18.127	-0.061	
14	-1.055	-0.364	0.302	-142.273	4.822	-3.723	

119 samples = one recording

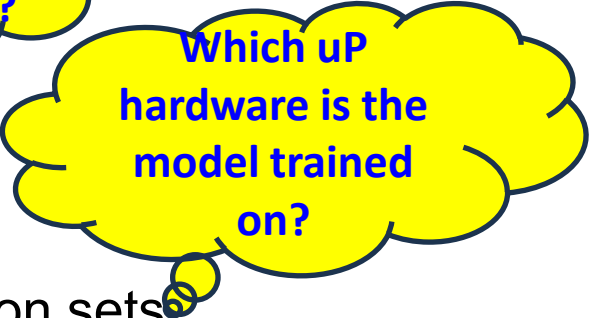
Training the model

- Steps involved in training the model
 - Parse the csv files and prepare the data to a format that is to be used to train a model based on fully connected neural network
 - **output** is either 'Punch' or 'Flex' (include unknown?)
 - use a **one-hot encoded** matrix for the output

Punch	Flex
1	0
0	1



What is
one-hot
encoding?



Which uP
hardware is the
model trained
on?

- Prepare the data for training
 - **Shuffle** the order of the inputs
 - Split the data into training, test and validation sets
(**60%:20%:20%**)

Building and Training the Model

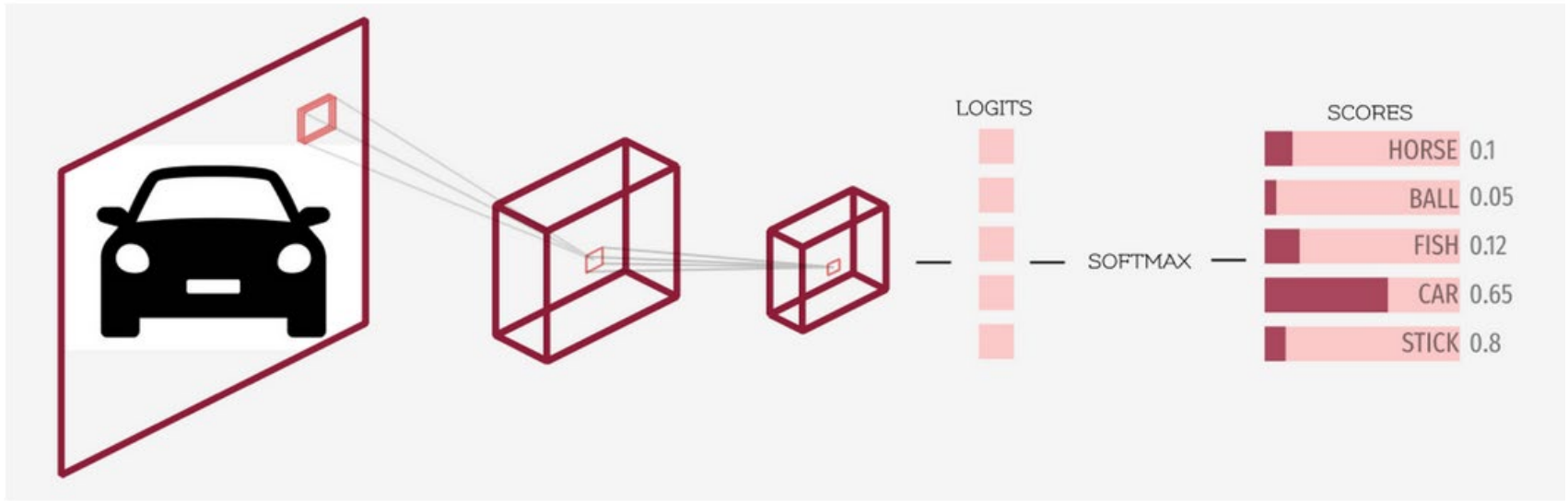
- Building the model - a sequential 3-layer model
 - `model = tf.keras.Sequential()`
 - `model.add(layers.Dense(50, activation='relu'))` // i.e. 50 neurons
 - `model.add(layers.Dense(15, activation='relu'))` // 15 neurons
 - `model.add(layers.Dense(NUM_GESTURES, activation='softmax'))`
- Training the model
 - `Optimizer = rmsprop`
 - `Loss function = mse`
 - `Metrics = mae`
 - `Epochs = 600`
 - `Batch = 1`
- `model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])`
- `model.fit(inputs_train, outputs_train, epochs=600, batch_size=1, validation_data=(inputs_validate, outputs_validate))`

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

σ = softmax
 $\vec{z}$ = input vector

softmax

Softmax



- Multi-classification problem.
- Outputs for each of the class are between 0 and 1 and the sum of these scores is equal to 1.

Data Normalisation

```
# read each csv file and push an input and output
for gesture_index in range(NUM_GESTURES):
    gesture = GESTURES[gesture_index]
    print(f"Processing index {gesture_index} for gesture '{gesture}'.")

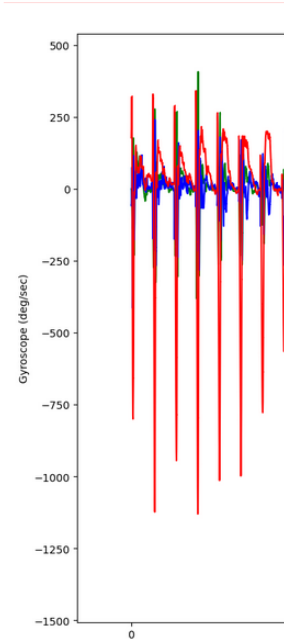
    output = ONE_HOT_ENCODED_GESTURES[gesture_index]
    df = pd.read_csv("content/" + gesture + ".csv")

    # calculate the number of gesture recordings in the file
    num_recordings = int(df.shape[0] / SAMPLES_PER_GESTURE)

    print(f"\tThere are {num_recordings} recordings of the {gesture} gesture.")

    for i in range(num_recordings):
        #print(i)
        tensor = []
        for j in range(SAMPLES_PER_GESTURE):
            index = i * SAMPLES_PER_GESTURE + j
            # normalize the input data, between 0 to 1:
            # - acceleration is between: -4 to +4
            # - gyroscope is between: -2000 to +2000
            tensor += [(df['aX'][index] + 4) / 8,
                      (df['aY'][index] + 4) / 8,
                      (df['aZ'][index] + 4) / 8,
                      (df['gX'][index] + 1500) / 2000,
                      (df['gY'][index] + 1500) / 2000,
                      (df['gZ'][index] + 1500) / 2000]

        inputs.append(tensor)
        outputs.append(output)
```



1D or 2D tensor?
Input tensor
shape?

Verifying the Model Performance

- // Graphing the History of Training

```
loss = history.history['loss']  
val_loss = history.history['val_loss']  
epochs = range(1, len(loss) + 1)  
:
```

- // Zoom in the convergence region of the model

```
SKIP = 100  
plt.plot(epochs[SKIP:], loss[SKIP:], 'g.', label='Training loss')  
:
```

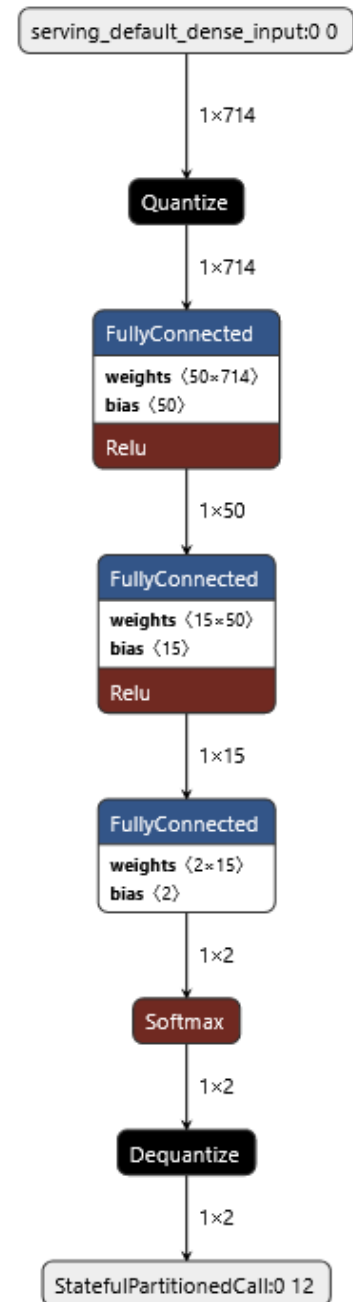
- // Check the mean absolute error

```
mae = history.history['mae']  
val_mae = history.history['val_mae']  
:
```

- // Verify the model using the test data

Gesture Model

- TFLite model of the Gesture Recognition Model
- 119 data points in one sample
- Each data point consist of data from 3-axis Accelerometer and 3-Axis Gyro data.
- $119 \times 6 = 714$.
- 50 samples collected for each gesture.



Verifying using Test Data

- Convert the trained model to TensorFlow Lite

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()
```

- Converting model to C file
- Integrate model with a TinyML program for Arduino - [IMU_Classifier.ino](#)
- Detail of Python code can be found at
 - https://colab.research.google.com/github/arduino/ArduinoTensorFlowLiteTutorials/blob/master/GestureToEmoji/arduino_tinyml_workshop.ipynb#scrollTo=0Xn1-Rn9Cp_8
- or link from
 - <https://blog.tensorflow.org/2019/11/how-to-get-started-with-machine.html>

IMU_Classifier Code snippet

```
// global variables used for TensorFlow Lite (Micro)
tflite::MicroErrorReporter tflErrorReporter; ← Error Log

// pull in all the TFLM ops, you can remove this line and
// only pull in the TFLM ops you need, if would like to reduce
// the compiled size of the sketch.
tflite::AllOpsResolver tflOpsResolver; ← TFLite Operations

const tflite::Model* tflModel = nullptr;
tflite::MicroInterpreter* tflInterpreter = nullptr;
TfLiteTensor* tflInputTensor = nullptr;
TfLiteTensor* tflOutputTensor = nullptr;

// Create a static memory buffer for TFLM, the size may need to
// be adjusted based on the model you are using
constexpr int tensorArenaSize = 8 * 1024; ← Memory allocation
byte tensorArena[tensorArenaSize] __attribute__((aligned(16)));

// array to map gesture index to a name
const char* GESTURES[] = { ← Gesture label
    "punch",
    "flex"
};
```

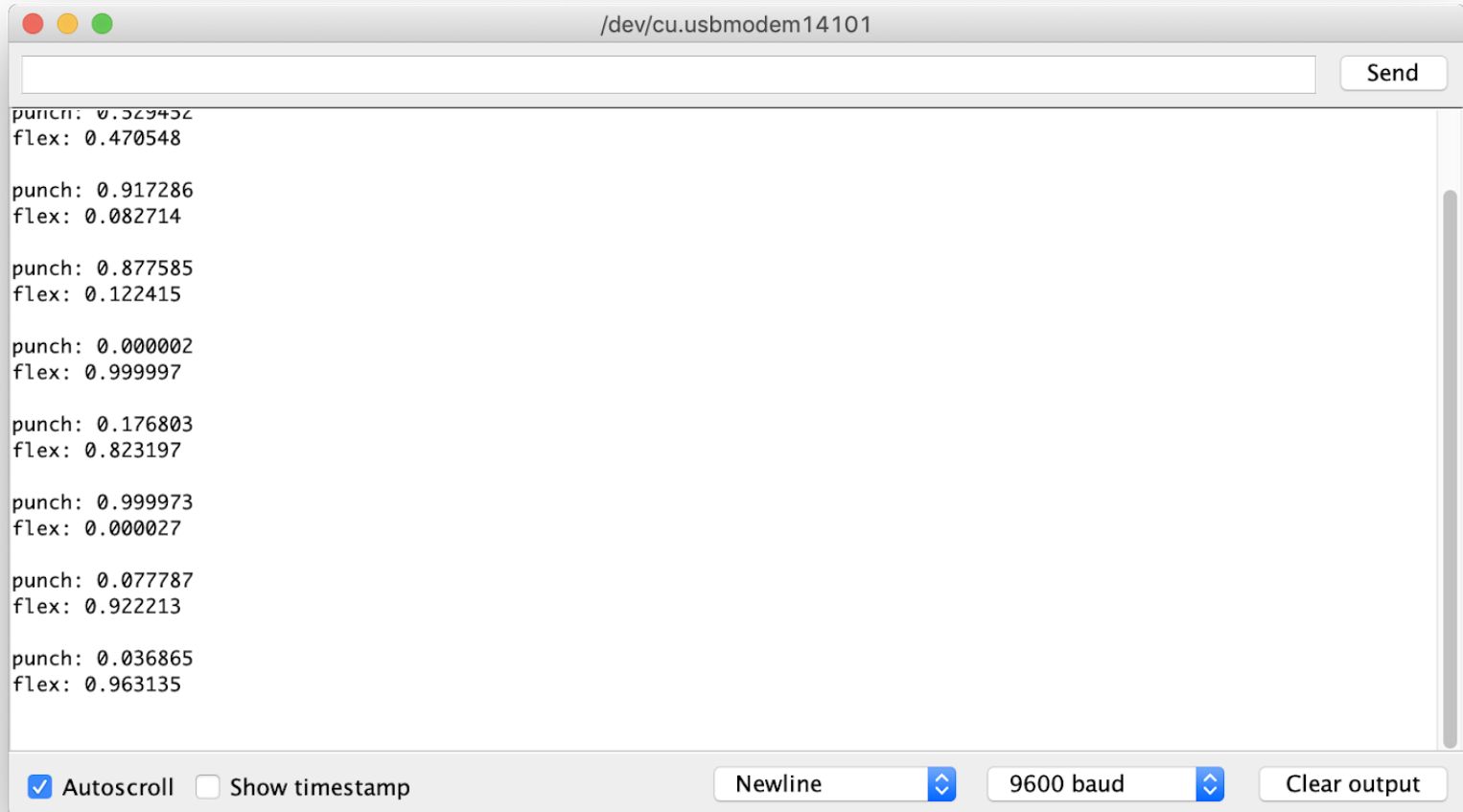
IMU_Classifier Code snippet

```
void setup() {  
    Serial.begin(9600);  
    while (!Serial);  
  
    // initialize the IMU ← Initialise IMU  
    if (!IMU.begin()) {  
        Serial.println("Failed to initialize IMU!");  
        while (1);  
    }  
  
    // get the TFL representation of the model byte array  
    tflModel = tflite::GetModel(gesture_model); ← Load Model  
    if (tflModel->version() != TFLITE_SCHEMA_VERSION) { ← Check Schema Vers  
        Serial.println("Model schema mismatch!");  
        while (1);  
    }  
  
    // Create an interpreter to run the model ← Create Interpreter  
    tflInterpreter = new tflite::MicroInterpreter(tflModel, tflOpsResolver, tensorAr  
  
    // Allocate memory for the model's input and output tensors  
    tflInterpreter->AllocateTensors(); ← Allocate Tensors in memory arena  
  
    // Get pointers for the model's input and output tensors  
    tflInputTensor = tflInterpreter->input(0);  
    tflOutputTensor = tflInterpreter->output(0);  
}
```


IMU_Classifier Code snippet

```
void loop() {  
  // Read data  
  IMU.readAcceleration(aX, aY, aZ);  
  IMU.readGyroscope(gX, gY, gZ);  
  // collect and store data in input tensor  
  tflInputTensor->data ...  
  // Run inference once sufficient data are collected (119 samples)  
  TfLiteStatus invokeStatus = tflInterpreter->Invoke();  
  
  // Send the output tensor values (confidence level of  
  // each gesture to the output  
  Serial.println(tflOutputTensor->data . . .  
}
```

IMU_Classifier Serial Monitor Output



END